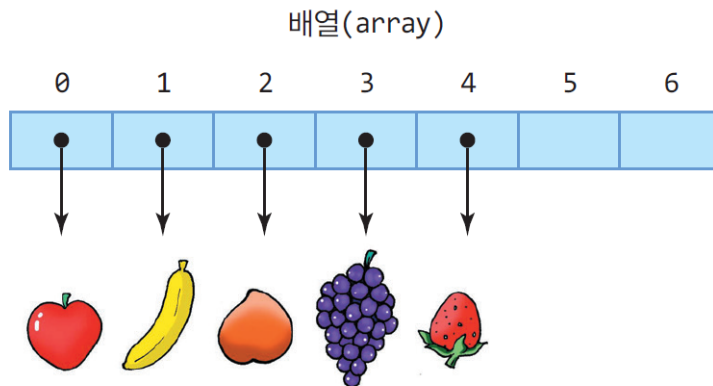


컬렉션(collection)의 개념

1

□ 컬렉션

- 요소(element) 객체들의 저장소
 - 객체들의 컨테이너라고도 불림
 - 요소의 개수에 따라 크기 자동 조절
 - 요소의 삽입, 삭제에 따른 요소의 위치 자동 이동
- 고정 크기의 배열을 다루는 어려움 해소
- 다양한 객체들의 삽입, 삭제, 검색 등의 관리 용이
- **배열과의 차이**



- 고정 크기 이상의 객체를 관리할 수 없다.
- 배열의 중간에 객체가 삭제되면 응용프로그램에서 자리를 옮겨야 한다.

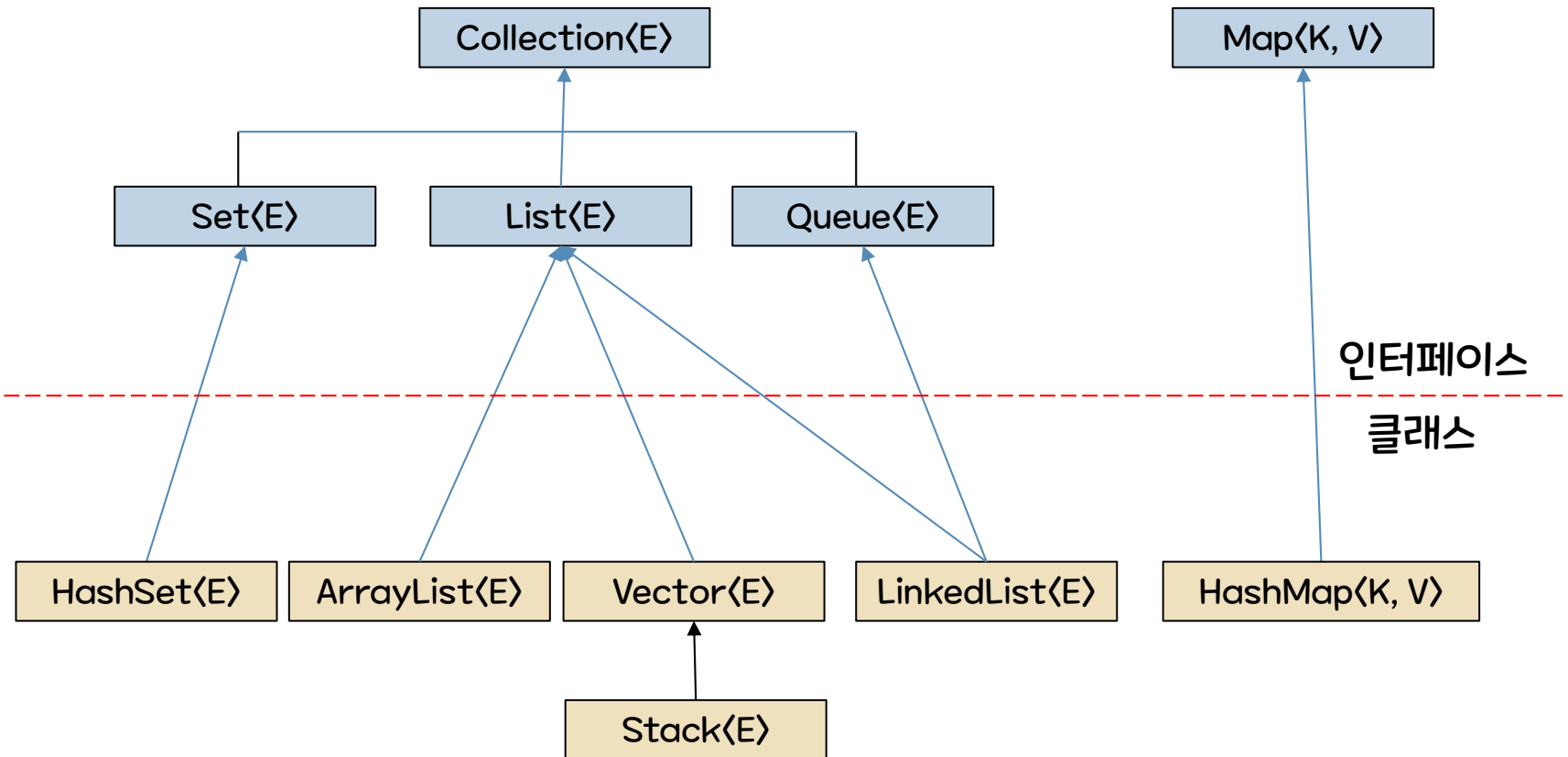
컬렉션(collection)



- 가변 크기로서 객체의 개수를 염려할 필요 없다.
- 컬렉션 내의 한 객체가 삭제되면 컬렉션이 자동으로 자리를 옮겨준다.

컬렉션을 위한 자바 인터페이스와 클래스

2



컬렉션과 제네릭

3

- 컬렉션은 제네릭(generics) 기법으로 구현됨
- 컬렉션의 요소는 객체만 가능
 - ▣ 기본적으로 int, char, double 등의 기본 타입 사용 불가
 - JDK 1.5부터 자동 박싱/언박싱으로 기본 타입 값을 객체로 자동 변환
- 제네릭
 - ▣ 특정 타입만 다루지 않고, 여러 종류의 타입으로 변신할 수 있도록 클래스나 메소드를 일반화시키는 기법
 - $\langle E \rangle$, $\langle K \rangle$, $\langle V \rangle$: 타입 매개 변수
 - 요소 타입을 일반화한 타입
 - ▣ 제네릭 클래스 사례
 - 제네릭 스택 : $\text{Stack}\langle E \rangle$
 - E에 특정 타입으로 구체화
 - 정수만 다루는 스택 $\text{Stack}\langle \text{Integer} \rangle$
 - 문자열만 다루는 스택 $\text{Stack}\langle \text{String} \rangle$

Vector<E>

4

□ Vector<E>의 특성

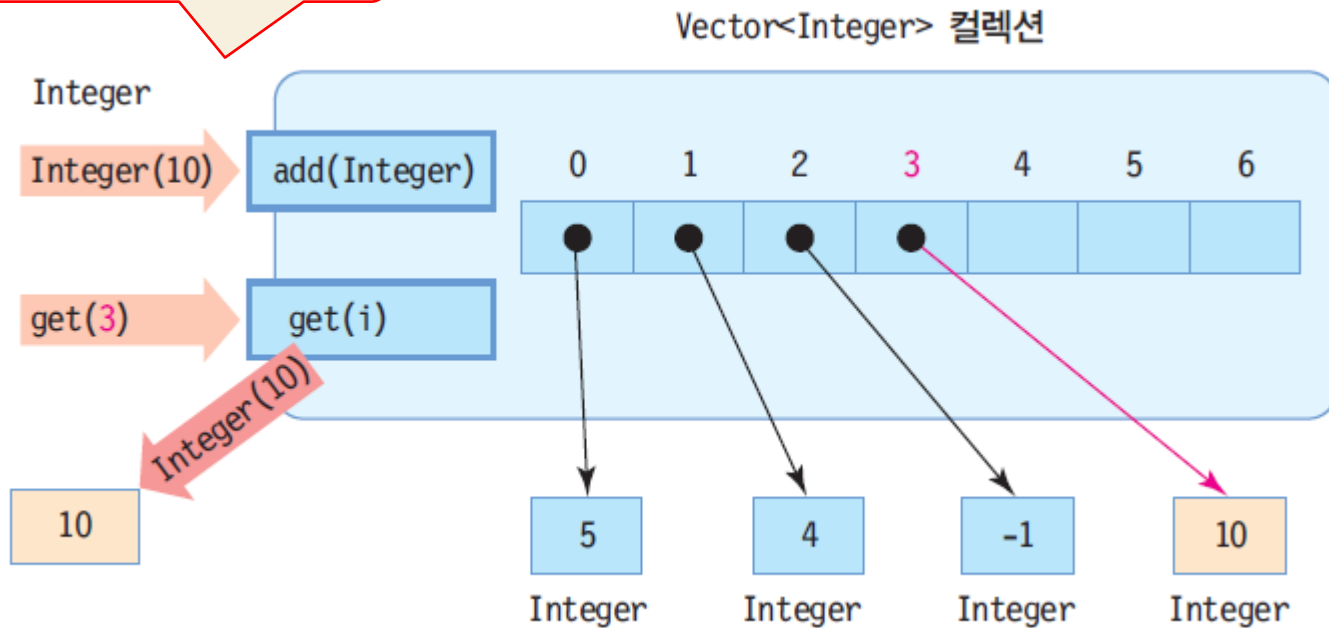
- ▣ java.util.Vector
 - <E>에서 E 대신 요소로 사용할 특정 타입으로 구체화
- ▣ 여러 객체들을 삽입, 삭제, 검색하는 컨테이너 클래스
 - 배열의 길이 제한 극복
 - 원소의 개수가 넘쳐나면 자동으로 길이 조절
- ▣ Vector에 삽입 가능한 것
 - 객체, null
 - 기본 타입은 Wrapper 객체로 만들어 저장
- ▣ Vector에 객체 삽입
 - 벡터의 맨 뒤에 객체 추가
 - 벡터 중간에 객체 삽입
- ▣ Vector에서 객체 삭제
 - 임의의 위치에 있는 객체 삭제 가능 : 객체 삭제 후 자동 자리 이동

Vector<Integer> 컬렉션 내부 구성

5

```
Vector<Integer> v = new Vector<Integer>();
```

add()를 이용하여 요소를 삽입하고 get()을
이용하여 요소를 검색합니다



타입 매개 변수 사용하지 않는 경우 경고 발생

6

Vector<Integer>로 타입 매개 변수를 사용하여야 함

예제 및 그림 소스 - chap07-ex01/src/VectorEx.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

*VectorEx.java

```
1 import java.util.Vector;
2
3 public class VectorEx {
4     public static void main(String[] args) {
5         Vector = new Vector(); // 정수 값만 다루는 벡터 생성
6     }
7 }
```

Vector is a raw type. References to generic type Vector<E> should be parameterized

4 quick fixes available:

- [Infer Generic Type Arguments...](#)
- [@ Add @SuppressWarnings 'rawtypes' to 'v'](#)
- [@ Add @SuppressWarnings 'rawtypes' to 'main\(\)'](#)
- [Configure problem severity](#)

Press 'F2' for focus

Vector<>를 사용하지 않으면 경고 발생

Writable Smart Insert 5 : 15 [6]

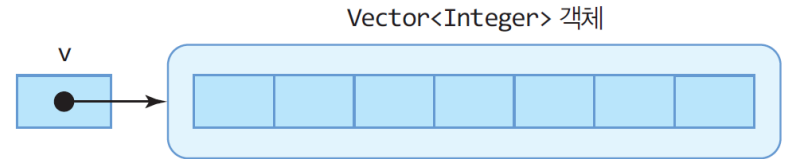
Vector<E> 클래스의 주요 메소드

7

메소드	설명
<code>boolean add(E element)</code>	벡터의 맨 뒤에 element 추가
<code>void add(int index, E element)</code>	인덱스 index에 element를 삽입
<code>int capacity()</code>	벡터의 현재 용량 리턴
<code>boolean addAll(Collection<? extends E> c)</code>	컬렉션 c의 모든 요소를 벡터의 맨 뒤에 추가
<code>void clear()</code>	벡터의 모든 요소 삭제
<code>boolean contains(Object o)</code>	벡터가 지정된 객체 o를 포함하고 있으면 true 리턴
<code>E elementAt(int index)</code>	인덱스 index의 요소 리턴
<code>E get(int index)</code>	인덱스 index의 요소 리턴
<code>int indexOf(Object o)</code>	o와 같은 첫 번째 요소의 인덱스 리턴, 없으면 -1 리턴
<code>boolean isEmpty()</code>	벡터가 비어 있으면 true 리턴
<code>E remove(int index)</code>	인덱스 index의 요소 삭제
<code>boolean remove(Object o)</code>	객체 o와 같은 첫 번째 요소를 벡터에서 삭제
<code>void removeAllElements()</code>	벡터의 모든 요소를 삭제하고 크기를 0으로 만들
<code>int size()</code>	벡터가 포함하는 요소의 개수 리턴
<code>Object[] toArray()</code>	벡터의 모든 요소를 포함하는 배열 리턴

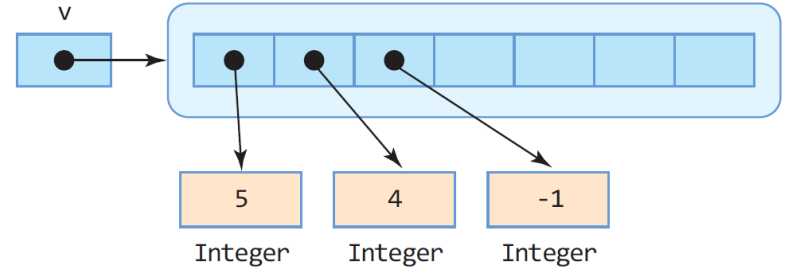
벡터 생성

```
Vector<Integer> v = new Vector<Integer>(7);
```



요소 삽입

```
v.add(5);  
v.add(4);  
v.add(-1);
```



요소 개수 n
벡터의 용량 c

```
int n = v.size(); // n은 3  
int c = v.capacity(); // c는 7
```

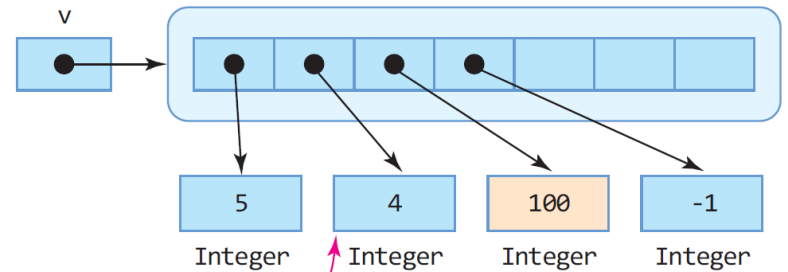
n = 3
c = 7

요소 중간 삽입

```
v.add(2, 100);
```

오류

```
v.add(5, 100);  
// v.size()보다 큰 곳에 삽입 불가능, 오류
```



요소 얻어내기

```
Integer obj = v.get(1);  
int i = obj.intValue();
```



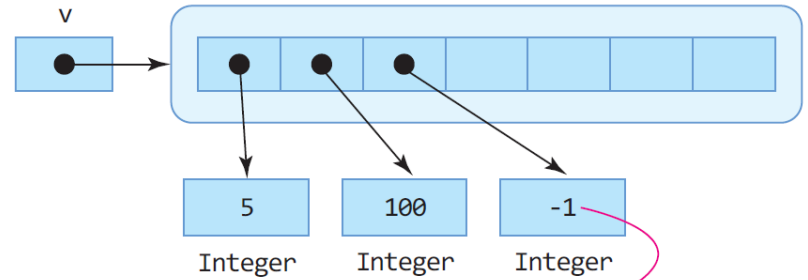
요소 삭제

```
v.remove(1);
```

오류

```
v.remove(4);
```

// 인덱스 4에 요소 객체가 없으므로 오류



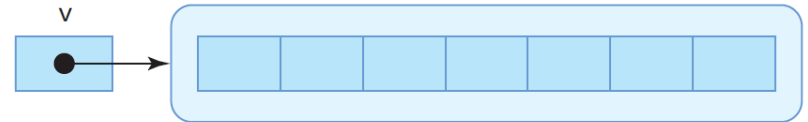
`last = -1`

마지막 요소

```
int last = v.lastElement();
```

모든 요소 삭제

```
v.removeAllElements();
```



컬렉션과 자동 박싱/언박싱

10

□ JDK 1.5 이전

- 기본 타입 데이터를 Wrapper 클래스를 이용하여 객체로 만들어 사용

```
Vector<Integer> v = new Vector<Integer>();  
v.add(Integer.valueOf(4));
```

- 컬렉션으로부터 요소를 얻어올 때, Wrapper 클래스로 캐스팅 필요

```
Integer n = (Integer)v.get(0);  
int k = n.intValue(); // k = 4
```

□ JDK 1.5부터

- 자동 박싱/언박싱이 작동하여 기본 타입 값 사용 가능

```
Vector<Integer> v = new Vector<Integer> ();  
v.add(4); // 4 → Integer.valueOf(4)로 자동 박싱  
int k = v.get(0); // Integer 타입이 int 타입으로 자동 언박싱, k = 4
```

- 제네릭의 타입 매개 변수를 기본 타입으로 구체화할 수는 없음



```
Vector<int> v = new Vector<int> (); // 오류
```

Wrapper 객체 생성

11

□ 기본 타입의 값으로 Wrapper 객체 생성

```
Integer i = Integer.valueOf(10);  
Character c = Character.valueOf('c');  
Double f = Double.valueOf(3.14);  
Boolean b = Boolean.valueOf(true);
```

```
Integer i = new Integer(10);  
Character c = new Character('c');  
Double f = new Double(3.14);  
Boolean b = new Boolean(true);
```

Java 9부터 생성자를 이용한
Wrapper 객체 생성 폐기

□ 문자열로 Wrapper 객체 생성

```
Integer i = Integer.valueOf("10");  
Double d = Double.valueOf("3.14");  
Boolean b = Boolean.valueOf("false");
```

```
Integer i = new Integer("10");  
Double d = new Double("3.14");  
Boolean b = new Boolean("false");
```

□ Float 객체는 double 타입의 값으로 생성 가능

```
Float f = Float.valueOf((double) 3.14);
```

주요 메소드

12

- Wrapper 객체들은 거의 유사, 많은 메소드가 static 타입
- Integer 클래스의 주요 메소드

메소드	설명
<code>static int bitCount(int i)</code>	정수 i의 이진수 표현에서 1의 개수 리턴
<code>float floatValue()</code>	float 타입으로 값 리턴
<code>int intValue()</code>	int 타입으로 값 리턴
<code>long longValue()</code>	long 타입으로 값 리턴
<code>short shortValue()</code>	short 타입으로 값 리턴
<code>static int parseInt(String s)</code>	문자열 s를 10진 정수로 변환한 값 리턴
<code>static int parseInt(String s, int radix)</code>	문자열 s를 지정된 진법의 정수로 변환한 값 리턴
<code>static String toBinaryString(int i)</code>	정수 i를 이진수 표현으로 변환한 문자열 리턴
<code>static String toHexString(int i)</code>	정수 i를 16진수 표현으로 변환한 문자열 리턴
<code>static String toOctalString(int i)</code>	정수 i를 8진수 표현으로 변환한 문자열 리턴
<code>static String toString(int i)</code>	정수 i를 문자열로 변환하여 리턴
<code>static Integer valueOf(int i)</code>	정수 i를 담은 Integer 객체 리턴
<code>static Integer valueOf(String s)</code>	문자열 s를 정수로 변환하여 담고 있는 Integer 객체 리턴

예제 7-1 : 정수만 다루는 Vector<Integer> 컬렉션 활용

13

정수만 다루는 벡터를 생성하고, 활용하는 기본 사례를 보인다.

```
import java.util.Vector;

public class VectorEx {
    public static void main(String[] args) {
        // 정수 값만 다루는 제네릭 벡터 생성
        Vector<Integer> v = new Vector<Integer>();

        v.add(5); // 5 삽입
        v.add(4); // 4 삽입
        v.add(-1); // -1 삽입

        // 벡터 중간에 삽입하기
        v.add(2, 100); // 4와 -1 사이에 정수 100 삽입

        System.out.println("벡터 내의 요소 객체 수 : " + v.size());
        System.out.println("벡터의 현재 용량 : " + v.capacity());

        // 모든 요소 정수 출력하기
        for(int i=0; i<v.size(); i++) {
            int n = v.get(i);
            System.out.println(n);
        }
    }
}
```

```
        // 벡터 속의 모든 정수 더하기
        int sum = 0;
        for(int i=0; i<v.size(); i++) {
            int n = v.elementAt(i);
            sum += n;
        }
        System.out.println("벡터에 있는 정수 합 : “
                                + sum);
    }
}
```

벡터 내의 요소 객체 수 : 4
벡터의 현재 용량 : 10
5
4
100
-1
벡터에 있는 정수 합 : 108

예제 7-2 : Point 클래스만 다루는 Vector<Point> 컬렉션 활용

14

점 (x, y)를 표현하는 Point 클래스를 만들고, Point의 객체만 다루는 벡터를 작성하라.

```
import java.util.Vector;
```

```
class Point {  
    private int x, y;  
  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public String toString() {  
        return "(" + x + "," + y + ")";  
    }  
}
```

```
public class PointVectorEx {  
    public static void main(String[] args) {  
        // Point 객체를 요소로만 가지는 벡터 생성  
        Vector<Point> v = new Vector<Point>();  
  
        // 3 개의 Point 객체 삽입  
        v.add(new Point(2, 3));  
        v.add(new Point(-5, 20));  
        v.add(new Point(30, -8));  
  
        v.remove(1); // 인덱스 1의 Point(-5, 20) 객체 삭제  
  
        // 벡터에 있는 Point 객체 모두 검색하여 출력  
        for(int i=0; i<v.size(); i++) {  
            Point p = v.get(i); // 벡터에서 i 번째 Point 객체 얻어내기  
            System.out.println(p); // p.toString()을 이용하여 객체 p 출력  
        }  
    }  
}
```

(2,3)
(30,-8)

컬렉션을 매개변수로 받는 메소드 만들기

15

- 컬렉션을 매개변수로 받는 메소드의 원형
 - ▣ 예) `public void printVector(Vector<Integer> v)`

```
// Integer 벡터를 매개변수로 받아 원소를 모두 출력하는 printVector() 메소드

public void printVector(Vector<Integer> v) {
    for(int i=0; i<v.size(); i++) {
        int n = v.get(i); // 벡터의 i 번째 정수
        System.out.println(n);
    }
}
```

```
Vector<Integer> v = new Vector<Integer>(); // Integer 벡터 생성
printVector(v); // 메소드 호출
```

자바의 타입 추론 기능의 진화

16

□ Java 7 이전

```
Vector<Integer> v = new Vector<Integer>(); // Java 7 이전
```

□ Java 7 이후

- ▣ 컴파일러의 타입 추론 기능 추가
- ▣ <>(다이아몬드 연산자)에 타입 매개변수 생략

```
Vector<Integer> v = new Vector<>(); // Java 7부터 추가, 가능
```

□ Java 10 이후

- ▣ var 키워드 도입, 컴파일러의 지역 변수 타입 추론 가능

```
var v = new Vector<Integer>(); // Java 10부터 추가, 가능
```


ArrayList<E>

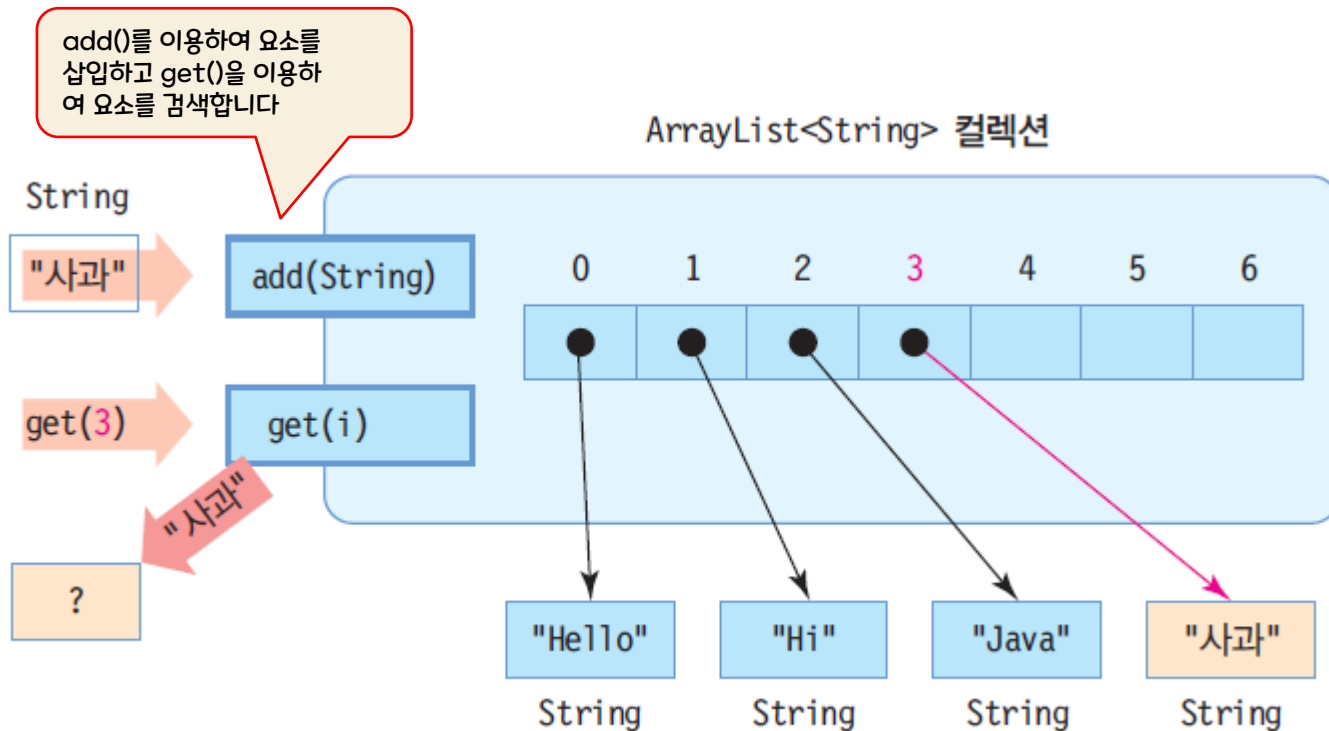
17

- ArrayList<E>의 특성
 - ▣ java.util.ArrayList, 가변 크기 배열을 구현한 클래스
 - <E>에서 E 대신 요소로 사용할 특정 타입으로 구체화
 - ▣ ArrayList에 삽입 가능한 것
 - 객체, null
 - 기본 타입은 박싱/언박싱으로 Wrapper 객체로 만들어 저장
 - ▣ ArrayList에 객체 삽입/삭제
 - 리스트의 맨 뒤에 객체 추가
 - 리스트의 중간에 객체 삽입
 - 임의의 위치에 있는 객체 삭제 가능
 - ▣ **Vector 클래스와 대부분의 기능 동일하나 스레드 동기화 기능 없음**
 - 다수 스레드가 동시에 ArrayList에 접근할 때 동기화되지 않음
 - 개발자가 스레드 동기화 코드 작성

ArrayList<String> 컬렉션의 내부 구성

18

```
ArrayList<String> al = new  
ArrayList<String>();
```



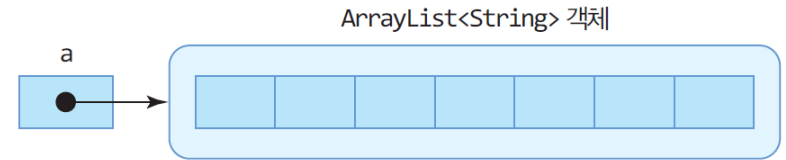
ArrayList<E> 클래스의 주요 메소드

19

메소드	설명
<code>boolean add(E element)</code>	ArrayList의 맨 뒤에 element 추가
<code>void add(int index, E element)</code>	인덱스 index 위치에 element 삽입
<code>boolean addAll(Collection<? extends E> c)</code>	컬렉션 c의 모든 요소를 ArrayList의 맨 뒤에 추가
<code>void clear()</code>	ArrayList의 모든 요소 삭제
<code>boolean contains(Object o)</code>	ArrayList가 지정된 객체를 포함하고 있으면 true 리턴
<code>E elementAt(int index)</code>	index 인덱스의 요소 리턴
<code>E get(int index)</code>	index 인덱스의 요소 리턴
<code>int indexOf(Object o)</code>	o와 같은 첫 번째 요소의 인덱스 리턴, 없으면 -1 리턴
<code>boolean isEmpty()</code>	ArrayList가 비어있으면 true 리턴
<code>E remove(int index)</code>	index 인덱스의 요소 삭제
<code>boolean remove(Object o)</code>	o와 같은 첫 번째 요소를 ArrayList에서 삭제
<code>int size()</code>	ArrayList가 포함하는 요소의 개수 리턴
<code>Object[] toArray()</code>	ArrayList의 모든 요소를 포함하는 배열 리턴

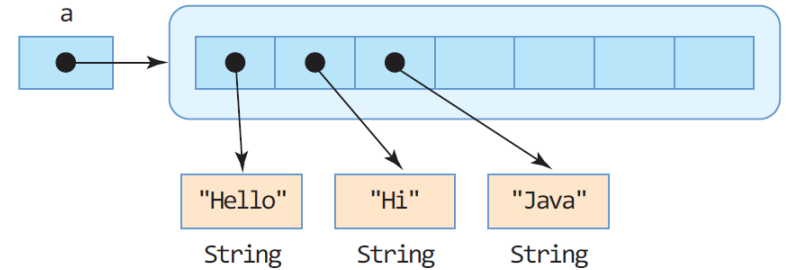
ArrayList 생성

```
ArrayList<String> a = new ArrayList<String>(7);
```



요소 삽입

```
a.add("Hello");  
a.add("Hi");  
a.add("Java");
```



요소 개수 n
용량

```
int n = a.size(); // n은 3  
int c = a.capacity(); // capacity() 메소드 없음
```

오류

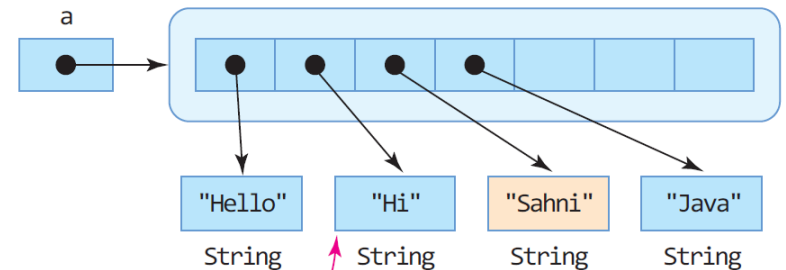
n = 3

요소 중간 삽입

```
a.add(2, "Sahni");
```

오류

```
a.add(5, "Sahni");  
// a.size()보다 큰 위치에 삽입 불가능, 오류
```



요소 알아내기

```
String str = a.get(1);
```

"Hi"

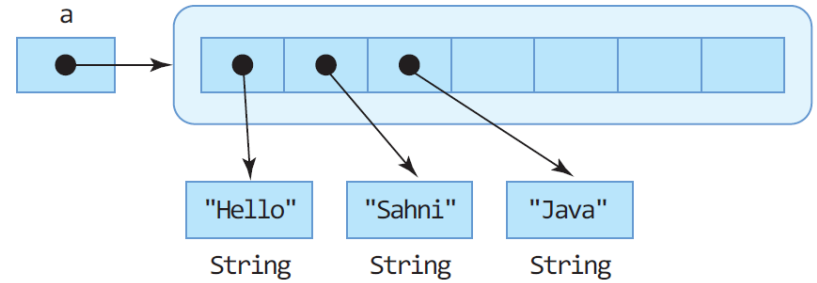


요소 삭제

```
a.remove(1);
```

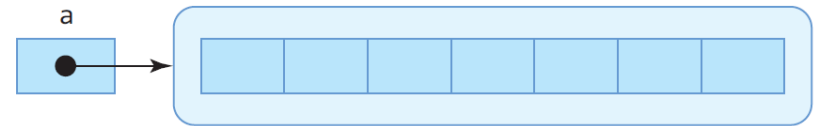
오류

```
a.remove(4); // 오류
```



모든 요소 삭제

```
a.clear();
```



예제 7-3 : 문자열 입력받아 ArrayList에 저장

22

이름을 4개 입력받아 ArrayList에 저장하고 모두 출력한 후 제일 긴 이름을 출력하라.

```
import java.util.*;

public class ArrayListEx {
    public static void main(String[] args) {
        // 문자열만 삽입가능한 ArrayList 컬렉션 생성
        ArrayList<String> a = new ArrayList<String>();

        // 키보드로부터 4개의 이름 입력받아 ArrayList에 삽입
        Scanner scanner = new Scanner(System.in);
        for(int i=0; i<4; i++) {
            System.out.print("이름을 입력하세요>>");
            String s = scanner.next(); // 키보드로부터 이름 입력
            a.add(s); // ArrayList 컬렉션에 삽입
        }

        // ArrayList에 들어 있는 모든 이름 출력
        for(int i=0; i<a.size(); i++) {
            // ArrayList의 i 번째 문자열 얻어오기
            String name = a.get(i);
            System.out.print(name + " ");
        }
    }
}
```

```
// 가장 긴 이름 출력
int longestIndex = 0;
for(int i=1; i<a.size(); i++) {
    if(a.get(longestIndex).length()
< a.get(i).length())
        longestIndex = i;
}
System.out.println("\n가장 긴 이름은 : " +
    a.get(longestIndex));
scanner.close();
}
```

```
이름을 입력하세요>>Mike
이름을 입력하세요>>Jane
이름을 입력하세요>>Ashley
이름을 입력하세요>>Helen
Mike Jane Ashley Helen
가장 긴 이름은 : Ashley
```

ArrayList<String> a = new ArrayList<>(); 나
var a = new ArrayList<String>(); 모두 가능

컬렉션의 순차 검색을 위한 Iterator

23

□ Iterator<E> 인터페이스

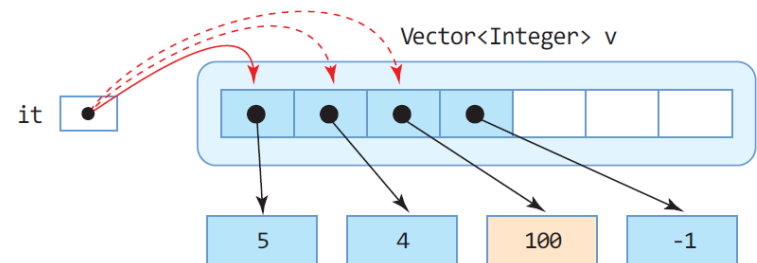
- ▣ Vector<E>, ArrayList<E>, LinkedList<E>가 상속받는 인터페이스
 - 리스트 구조의 컬렉션에서 요소의 순차 검색을 위한 메소드 포함
- ▣ Iterator<E> 인터페이스 메소드

메소드	설명
boolean hasNext()	방문할 요소가 남아 있으면 true 리턴
E next()	다음 요소 리턴
void remove()	마지막으로 리턴된 요소 제거

▣ iterator() 메소드 : Iterator 객체 반환

- Iterator 객체를 이용하여 인덱스 없이 순차적 검색 가능

```
Vector<Integer> v = new Vector<Integer>();  
Iterator<Integer> it = v.iterator();  
while(it.hasNext()) { // 모든 요소 방문  
    int n = it.next(); // 다음 요소 리턴  
    ...  
}
```



예제 7-4 : Iterator를 이용하여 Vector의 모든 요소를 출력하고 합 구하기

24

예제 7-1의 코드를 Iterator<Integer>를 이용하여 수정하라.

```
import java.util.*;

public class IteratorEx {
    public static void main(String[] args) {
        // 정수 값만 다루는 제네릭 벡터 생성
        Vector<Integer> v = new Vector<Integer>();
        v.add(5); // 5 삽입
        v.add(4); // 4 삽입
        v.add(-1); // -1 삽입
        v.add(2, 100); // 4와 -1 사이에 정수 100 삽입

        // Iterator를 이용한 모든 정수 출력하기
        Iterator<Integer> it = v.iterator();
        while(it.hasNext()) {
            int n = it.next();
            System.out.println(n);
        }
    }
}
```

```
        // Iterator를 이용하여 모든 정수 더하기
        int sum = 0;
        it = v.iterator(); // Iterator 객체 얻기
        while(it.hasNext()) {
            int n = it.next();
            sum += n;
        }
        System.out.println("벡터에 있는 정수 합 : " + sum);
    }
}
```

```
5
4
100
-1
벡터에 있는 정수 합 : 108
```


HashMap<K,V>

25

□ HashMap<K,V>

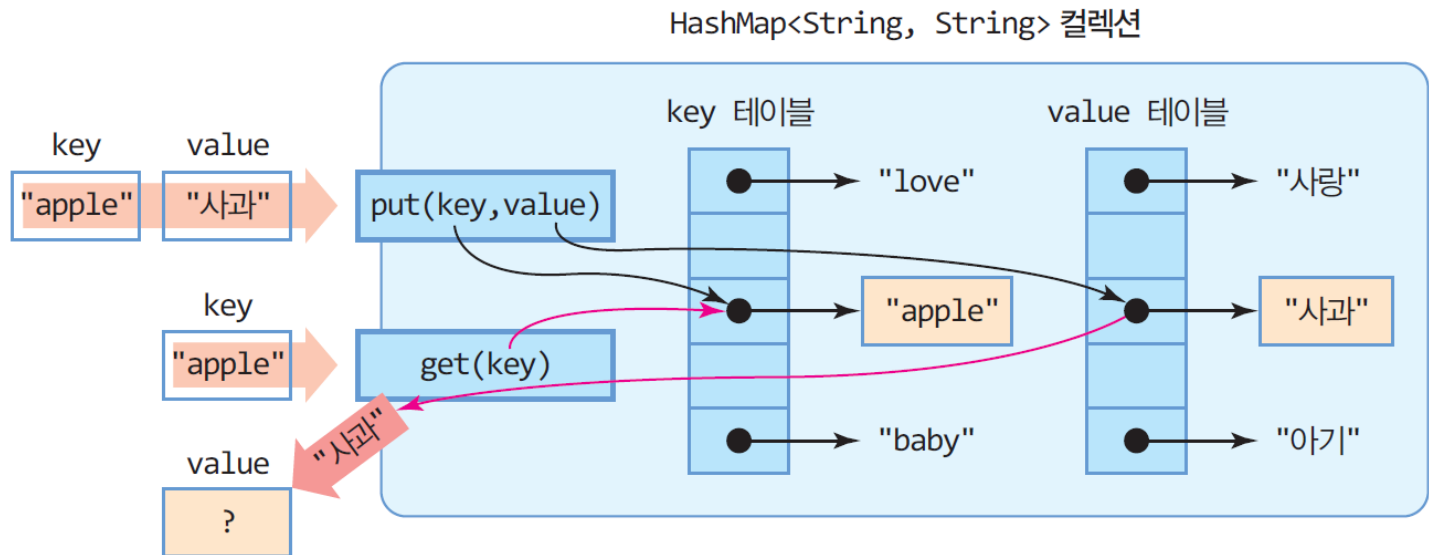
- 키(key)와 값(value)의 쌍으로 구성되는 요소를 다루는 컬렉션
 - java.util.HashMap
 - K는 키로 사용할 요소의 타입, V는 값으로 사용할 요소의 타입 지정
 - 키와 값이 한 쌍으로 삽입
 - 키는 해시맵에 삽입되는 위치 결정에 사용
 - 값을 검색하기 위해서는 반드시 키 이용
- 삽입, 삭제, 검색이 빠른 특징
 - 요소 삽입 : put() 메소드
 - 요소 검색 : get() 메소드
- 예) HashMap<String, String> 생성, 요소 삽입, 요소 검색

```
HashMap<String, String> h = new HashMap<String, String>();  
h.put("apple", "사과"); // "apple" 키와 "사과" 값의 쌍을 해시맵에 삽입  
String kor = h.get("apple"); // "apple" 키로 값 검색. kor는 "사과"
```

HashMap<String, String>의 내부 구성

26

```
HashMap<String, String> map = new HashMap<String, String>();
```



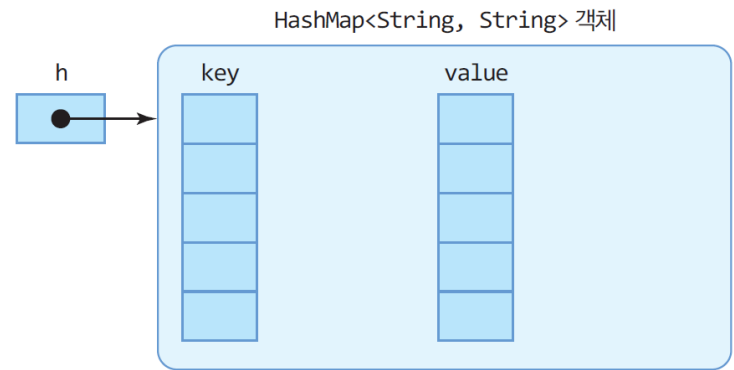
HashMap<K,V>의 주요 메소드

27

메소드	설명
<code>void clear()</code>	해시맵의 모든 요소 삭제
<code>boolean containsKey(Object key)</code>	지정된 키(key)를 포함하고 있으면 true 리턴
<code>boolean containsValue(Object value)</code>	지정된 값(value)에 일치하는 키가 있으면 true 리턴
<code>V get(Object key)</code>	지정된 키(key)의 값 리턴, 키가 없으면 null 리턴
<code>boolean isEmpty()</code>	해시맵이 비어 있으면 true 리턴
<code>Set<K> keySet()</code>	해시맵의 모든 키를 담은 Set<K> 컬렉션 리턴
<code>V put(K key, V value)</code>	key와 value 쌍을 해시맵에 저장
<code>V remove(Object key)</code>	지정된 키(key)를 찾아 키와 값 모두 삭제
<code>int size()</code>	HashMap에 포함된 요소의 개수 리턴

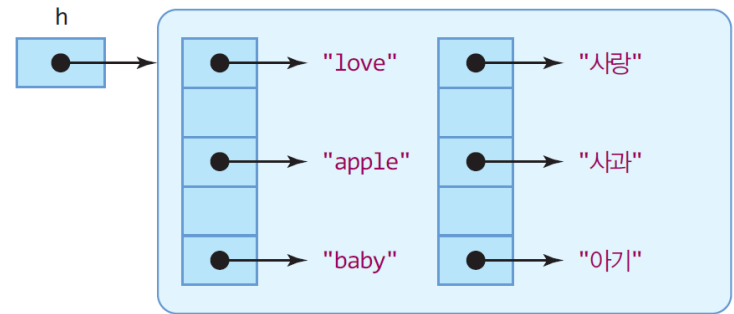
해시맵 생성

```
HashMap<String, String> h =  
new HashMap<String, String>();
```



(키, 값) 삽입

```
h.put("baby", "아기");  
h.put("love", "사랑");  
h.put("apple", "사과");
```



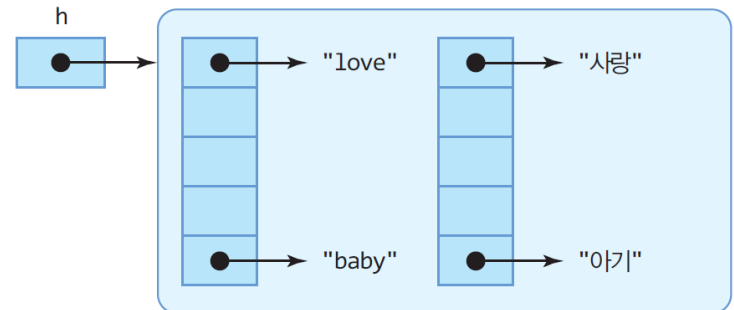
키로 값 읽기

```
String kor = h.get("love");
```

`kor = "사랑"`

키로 요소 삭제

```
h.remove("apple");
```



요소 개수

```
int n = h.size();
```

`n = 2`

예제 7-5 : HashMap을 이용하여 (영어, 한글) 단어 쌍의 저장 검색

29

(영어, 한글) 단어를 쌍으로 해시맵에 저장하고 영어로 한글을 검색하는 프로그램을 작성하라. "exit"이 입력되면 프로그램을 종료한다.

```
import java.util.*;

public class HashMapDicEx {
    public static void main(String[] args) {
        // 영어 단어와 한글 단어의 쌍을 저장하는 HashMap 컬렉션 생성
        HashMap<String, String> dic =
            new HashMap<String, String>();

        // 3 개의 (key, value) 쌍을 dic에 저장
        dic.put("baby", "아기"); // "baby"는 key, "아기"은 value
        dic.put("love", "사랑");
        dic.put("apple", "사과");

        // 영어 단어를 입력받고 한글 단어 검색. "exit" 입력받으면 종료
        Scanner scanner = new Scanner(System.in);
        while(true) {
            System.out.print("찾고 싶은 단어는?");
            String eng = scanner.next();
            if(eng.equals("exit")) {
                System.out.println("종료합니다...");
                break;
            }
        }
    }
}
```

```
// 해시맵에서 '키' eng의 '값' kor 검색
String kor = dic.get(eng);
if(kor == null)
    System.out.println(eng +
        "는 없는 단어 입니다.");
else
    System.out.println(kor);
}
scanner.close();
}
}
```

찾고 싶은 단어는?apple
사과
찾고 싶은 단어는?babo
babo는 없는 단어 입니다.
찾고 싶은 단어는?exit
종료합니다...

"babo"를 해시맵에서 찾을 수
없기 때문에 null 리턴

예제 7-6 HashMap을 이용하여 자바 과목의 이름과 점수 관리

30

해시맵을 이용하여 학생의 이름과 자바 점수를 기록 관리하는 프로그램을 작성하라

```
import java.util.*;

public class HashMapScoreEx {
    public static void main(String[] args) {
        // 사용자 이름과 점수를 기록하는 HashMap 컬렉션 생성
        HashMap<String, Integer> javaScore =
            new HashMap<String, Integer>();

        // 5 개의 점수 저장
        scoreMap.put("김성동", 97);
        scoreMap.put("황기태", 88);
        scoreMap.put("김남윤", 98);
        scoreMap.put("이재문", 70);
        scoreMap.put("한원선", 99);

        System.out.println("HashMap의 요소 개수 : "
            + javaScore.size());

        // 모든 사람의 점수 출력.
        // javaScore에 들어 있는 모든 (key, value) 쌍 출력
        // key 문자열을 가진 집합 Set 컬렉션 리턴
        Set<String> keys = javaScore.keySet();

        // key 문자열을 순서대로 접근할 수 있는 Iterator 리턴
        Iterator<String> it = keys.iterator();
```

```
        while(it.hasNext()) {
            String name = it.next();
            int score = javaScore.get(name);
            System.out.println(name + " : " + score);
        }
    }
}
```

HashMap의 요소 개수 :5
이재문 : 70
한원선 : 99
김남윤 : 98
김성동 : 97
황기태 : 88

예제 7-7 HashMap에 객체 저장, 학생 정보 관리

31

id와 전화번호로 구성되는 Student 클래스를 만들고, 이름을 '키'로 하고 Student 객체를 '값'으로 하는 해시맵을 작성하라.

```
import java.util.*;

class Student { // 학생을 표현하는 클래스
    int id;
    String tel;
    public Student(int id, String tel) {
        this.id = id; this.tel = tel;
    }
}
```

```
검색할 이름?이재문
id:2, 전화:010-222-2222
검색할 이름?김남윤
id:3, 전화:010-333-3333
검색할 이름?
```

```
public class HashMapStudentEx {
    public static void main(String[] args) {
        // 학생 이름과 Student 객체를 쌍으로 저장하는 HashMap 컬렉션 생성
        HashMap<String, Student> map = new HashMap<String, Student>();

        // 3 명의 학생 저장
        map.put("황기태", new Student(1, "010-111-1111"));
        map.put("이재문", new Student(2, "010-222-2222"));
        map.put("김남윤", new Student(3, "010-333-3333"));

        Scanner scanner = new Scanner(System.in);
        while(true) {
            System.out.print("검색할 이름?");
            String name = scanner.nextLine(); // 사용자로부터 이름 입력
            if(name.equals("exit"))
                break; // while 문을 벗어나 프로그램 종료
            Student student = map.get(name); // 이름에 해당하는 Student 객체 검색
            if(student == null)
                System.out.println(name + "은 없는 사람입니다.");
            else
                System.out.println("id:" + student.getId() + ", 전화:" + student.getTel());
        }
        scanner.close();
    }
}
```